Pasted text.txt
Document

page.tsx:1742 deleteActivity error Error: "<Activity: test>" needs to have a value for field "id" before this many-to-many relationship can be used.
    at postGraphQL (page.tsx:72:26)
    at async deleteActivityFromStructure (page.tsx:1730:7)
error @ intercept-console-error.ts:42
deleteActivityFromStructure @ page.tsx:1742
await in deleteActivityFromStructure
onConfirm @ page.tsx:2326
onConfirm @ page.tsx:3235
executeDispatch @ react-dom-client.development.js:20543
runWithFiberInDEV @ react-dom-client.development.js:986
processDispatchQueue @ react-dom-client.development.js:20593
(anonymous) @ react-dom-client.development.js:21164
batchedUpdates$1 @ react-dom-client.development.js:3377
dispatchEventForPluginEventSystem @ react-dom-client.development.js:20747
dispatchEvent @ react-dom-client.development.js:25693
dispatchDiscreteEvent @ react-dom-client.development.js:25661
<button>
exports.jsxDEV @ react-jsx-dev-runtime.development.js:342
ConfirmDialog @ ConfirmDialog.tsx:73
react_stack_bottom_frame @ react-dom-client.development.js:28038
renderWithHooksAgain @ react-dom-client.development.js:8084

renderWithHooks @ react-dom-client.development.js:7996
updateFunctionComponent @ react-dom-client.development.js:10501
beginWork @ react-dom-client.development.js:12136
runWithFiberInDEV @ react-dom-client.development.js:986
performUnitOfWork @ react-dom-client.development.js:18997
workLoopSync @ react-dom-client.development.js:18825
renderRootSync @ react-dom-client.development.js:18806
performWorkOnRoot @ react-dom-client.development.js:17835
performSyncWorkOnRoot @ react-dom-client.development.js:20399
flushSyncWorkAcrossRoots_impl @ react-dom-client.development.js:20241
processRootScheduleInMicrotask @ react-dom-client.development.js:20280
(anonymous) @ react-dom-client.development.js:20418Understand this error
forward-logs-shared.ts:95 [Fast Refresh] rebuilding
forward-logs-shared.ts:95 [Fast Refresh] done in 209ms
i see this response when i try to delete an activity
but when i click on delete again it detes activity sucessfully
i added my programmanagement file

I understand this error — and the "**first click fails, second click works**" behavior is actually a **big clue**.

What this error means

This backend error:

> `"<Activity: test>" needs to have a value for field "id" before this many-to-many relationship can be used.`

is a **Django ORM error**.

It happens when backend code tries to use a **ManyToMany field** on an `Activity` object **before that object has a primary key (`id`)**.

In simple terms:

- Django fetched or created an `Activity` instance like `<Activity: test>`
- But that instance is **unsaved** (or effectively treated as unsaved / pk is `None`)
- Then backend code did something like:
  - `activity.attachments.all()`
  - `activity.programs.set(...)`
  - `activity.some_m2m.clear()`
  - `activity.some_m2m.exists()`
- Django throws this exact error because **M2M cannot be used until the object has an id**

Why it fails first time, then works second time

Your frontend delete flow is:

1. **Detach activity from program/session/topic**
2. **Then call** `DELETE_ACTIVITY_MUTATION`

From your code in `deleteActivityFromStructure()` Pasted text:

- First it calls:
  - `UPDATE_PROGRAM_MUTATION` **or**
  - `UPDATE_SESSION_MUTATION` **or**
  - `UPDATE_TOPIC_MUTATION`
- Then it calls:

- `DELETE_ACTIVITY_MUTATION`

So on the **first click**:

- The **detach update likely succeeds**
- Then the **delete mutation fails** because backend delete logic touches an M2M on an invalid/unsaved `Activity` instance

That means:

- Activity is **removed from the structure**
- But **not actually deleted**

Then on the **second click**:

- Since it is already detached, the problematic relation state is different
- Now delete mutation can succeed (or the update step is effectively a no-op)
- So it finally deletes

✅ That is exactly why you see:

- **1st click → error**
- **2nd click → success**

Most likely backend cause (very likely)

The issue is **not primarily in your frontend**.

It is almost certainly in your `DELETE_ACTIVITY_MUTATION` **backend code** (or a signal triggered by it).

Most likely bad patterns in backend

Somewhere in delete mutation / signal / subscription code, you probably have something like:

```python
activity = Activity(name="test")   # unsaved instance
```

Python Run

or

```python
activity = Activity.objects.filter(id=id).first()
activity.delete()
```

Python Run

or

```python
activity = Activity.objects.get(id=id)
activity.delete()
```

Python Run

or

```python
instance = Activity(...)
# before instance.save()
```

Python Run

Very important: your own backend history strongly suggests this

Since you recently integrated **attachments with activities**, the most likely failing M2M is:

- `activity.attachments`
- or `activity.secure_attachments`
- or some assignment relation around attachments

That fits the exact error perfectly.

Where I would check first in backend

Please check these in your backend:

1) `DeleteActivity` mutation

Search for:

- `class DeleteActivity`
- `deleteActivity`
- `mutate(...)`

Look for any code that:

- fetches activity
- deletes activity
- then accesses any M2M field afterward

❌ Bad

```
activity = Activity.objects.get(id=id)
activity.delete()
```

Python Run

✅ Correct

```python
activity = Activity.objects.get(id=id)
files = list(activity.attachments.all())   # if needed before delete
```

Python Run

---

2) Signals on Activity

Check for:

- `@receiver(pre_delete, sender=Activity)`
- `@receiver(post_delete, sender=Activity)`
- `@receiver(post_save, sender=Activity)`

If inside those signals you do:

Python Run

or any M2M access on `instance`

that can cause this exact problem.

Important note:

In Django, **M2M relations are tricky in delete signals** and often unsafe depending on timing.

3) `send_subscription_and_notification(...)`

You imported this in earlier backend code.

If your delete mutation does something like:

```python
```

and that function accesses:

- `activity.attachments`
- `activity.program_set`
- any M2M relation

then this is likely the culprit.

Why your frontend code itself is mostly okay

Your delete flow here is structurally fine:

```typescript
await postGraphQL(UPDATE_*_MUTATION, updatePayload);
```

That logic is valid.

The only frontend issue is:

- If first call succeeds and second fails, your UI shows generic failure
- but backend state is now **partially changed**

So user sees confusing behavior.

Best fix (backend) ✅

Fix your `DELETE_ACTIVITY_MUTATION`

Safe pattern:

```
class DeleteActivity(graphene.Mutation):
    class Arguments:
        id = graphene.Int(required=True)

    success = graphene.Boolean()

    @classmethod
    def mutate(cls, root, info, id):
        activity = Activity.objects.get(id=id)

        # If you need related data, capture BEFORE delete
        attachment_ids = list(activity.attachments.values_list("id", flat=True))

        # delete
        activity.delete()

        # send notifications using primitive data, not activity.m2m after delete
        # send_subscription_and_notification(activity_id=id, attachment_ids=attachment_ids, mut
```

```
        return DeleteActivity(success=True)
```

Strong suspicion: your delete mutation accesses M2M after delete

This is the most probable exact pattern:

❌ Likely current code

```python
activity = Activity.objects.get(id=id)
activity.delete()
```

Python Run

✅ Fix

```python
activity = Activity.objects.get(id=id)

# serialize what you need BEFORE delete
activity_data = {
    "id": activity.id,
    "name": activity.name,
    "attachment_ids": list(activity.attachments.values_list("id", flat=True)),
}

activity.delete()

send_subscription_and_notification(activity_data, "DELETE")
```

Small frontend improvement (recommended)

Even after backend fix, I'd improve your frontend to avoid partial-success confusion.

Option 1: Delete first, then refresh

If your backend delete mutation already removes relations safely, then just do:

- `DELETE_ACTIVITY_MUTATION`
- refresh program

This is best **if backend delete fully handles cleanup**

Cleaner frontend:

TypeScript

Option 2: Keep current flow but show better debugging

Right now you catch everything together, so you don't know which mutation failed.

Split the two calls:

```
try {
  await postGraphQL(UPDATE_PROGRAM_MUTATION, updatePayload);
  console.log("Detach success");
```

```
    await postGraphQL(DELETE_ACTIVITY_MUTATION, { id: getIntFromString(activityId) });
    console.log("Delete success");
  } catch (err) {
    console.error("deleteActivity step failed", err);
  }
```

This will confirm exactly whether:

- detach succeeds
- delete fails

I'm almost sure that's what's happening.

My diagnosis in one line

Root cause:

Your backend delete mutation (or a signal/helper it calls) is touching an Activity many-to-many field after the Activity is deleted / before it has a valid pk.

Why second click works:

First click detaches the activity successfully but delete fails; second click retries delete when relation state has changed, so it succeeds.

What I need from you next (best next step)

Please send me **only these backend parts** and I'll tell you the exact line to fix:

1. `DELETE_ACTIVITY_MUTATION` GraphQL string from frontend (from `lib/queries`)
2. Backend `DeleteActivity` mutation class

3. Any `Activity` model M2M fields (especially attachments)
4. Any signals for `Activity`
5. `send_subscription_and_notification(...)` if delete uses it

Very likely quick fix for you right now

If your delete mutation currently does anything after `activity.delete()` with the same instance, change it immediately.

Rule:

✅ **Read related fields before delete**
❌ **Do not access M2M after delete**

If you want, send me your **backend** `DeleteActivity` **mutation code now** and I'll give you the **exact corrected code block ready to paste**.

Sources

from leaderboard.decorators import graphql_permission_required
import graphene
from django.utils.dateparse import parse_datetime
from django.contrib.auth import get_user_model

```
from ...models import Program, Activity,ACTIVITY_TYPE_CHOICES
from attachments.models import SecureAttachment
from ..types import ProgramActivityType
from leaderboard.events import send_subscription_and_notification
from django.db.models import Q
from django.db.models import Prefetch

User = get_user_model()

class CreateActivity(graphene.Mutation):
    class Arguments:
        type = graphene.String(required=True)
        name = graphene.String(required = True)
        template_id = graphene.Int()
        description = graphene.String()
        deadline = graphene.String()
        mapp_survey = graphene.Int()
        is_360 = graphene.Boolean()
        mapp_survey_name = graphene.String()
        mapp_tool = graphene.String()
        is_feedback = graphene.Boolean()
        attachment_ids = graphene.List(graphene.Int)

    activity = graphene.Field(ProgramActivityType)
    success = graphene.Boolean()
    message = graphene.String()

    @graphql_permission_required
    @staticmethod
```

```
    def mutate(root, info, type,
name,template_id=None,description=None, deadline=None,
mapp_survey=None, is_360=None,mapp_survey_name = None,
is_feedback=False,mapp_tool = None,attachment_ids=None):
        try:
            deadline_parsed = parse_datetime(deadline) if deadline else
None

            activity = Activity(
                type=type,
                name = name,
                template_id = template_id,
                description=description if description else '',
                deadline=deadline_parsed,
                mapp_survey=mapp_survey,
                is_360=is_360,
                mapp_survey_name = mapp_survey_name,
                is_feedback = is_feedback,
                mapp_tool = mapp_tool,
            )
            activity.full_clean()
            activity.save()
            if attachment_ids:
                attachments = SecureAttachment.objects.filter(
                    id__in=attachment_ids,
                    is_active=True
                )
                activity.attachments.set(attachments)
```

```
            send_subscription_and_notification(

subscription_module="program.graphql.Activity.subscriptions",
                subscription_class="ActivitySubscription",
                group=info.context.user.username,
                subscription_payload={
                    "object": activity,
                    "mutation": "CREATE",
                },
                # notification_event_type="survey_token_created",
                # notification_payload={
                #     "object": token,
                #     "user_id": user.id,
                # },
            )
            return CreateActivity(activity=activity, success=True,
message="Activity created successfully")
        except Program.DoesNotExist:
            return CreateActivity(activity=None, success=False,
message="Program not found.")
        except Exception as e:
            return CreateActivity(activity=None, success=False,
message=str(e))


class UpdateActivity(graphene.Mutation):
    class Arguments:
        id = graphene.ID(required=True)
        template_id = graphene.Int()
```

```
        name = graphene.String()
        type = graphene.String()
        description = graphene.String()
        deadline = graphene.String()  # ISO 8601 datetime string
        mapp_survey = graphene.Int()
        is_360 = graphene.Boolean()
        mapp_survey_name = graphene.String()
        is_feedback = graphene.Boolean()
        mapp_tool = graphene.String()
        attachment_ids = graphene.List(graphene.Int)

    activity = graphene.Field(ProgramActivityType)
    success = graphene.Boolean()
    message = graphene.String()

    @graphql_permission_required
    @staticmethod
    def mutate(root, info, id, template_id=None,name=None,
type=None, description=None, deadline=None, mapp_survey=None,
is_360=None,mapp_survey_name = None,
is_feedback=None,mapp_tool = None,attachment_ids=None):
        try:
            activity = Activity.objects.get(pk=id)

            # Handle type with choices validation
            if type is not None:
                activity.type = type

            # Handle deadline parsing
```

```
        if deadline is not None:
            deadline_parsed = parse_datetime(deadline)
            if not deadline_parsed:
                return UpdateActivity(activity=None, success=False,
message="Invalid deadline format.")
            activity.deadline = deadline_parsed

        # Update simple fields (None will set NULL)
        if name is not None:
            activity.name = name
        if description is not None:
            activity.description = description
        activity.mapp_survey = mapp_survey
        activity.mapp_survey_name = mapp_survey_name
        activity.mapp_tool = mapp_tool
        activity.is_360 = is_360
        if template_id:
            activity.template_id = template_id

        if is_feedback is not None:
            activity.is_feedback = is_feedback
        activity.full_clean()
        activity.save()
        if attachment_ids is not None:
            attachments = SecureAttachment.objects.filter(
                id__in=attachment_ids,
                is_active=True
            )
            activity.attachments.set(attachments)
```

```
        send_subscription_and_notification(

subscription_module="program.graphql.Activity.subscriptions",
            subscription_class="ActivitySubscription",
            group=info.context.user.username,
            subscription_payload={
                "object": activity,
                "mutation": "UPDATE",
            },
            # notification_event_type="survey_token_created",
            # notification_payload={
            #     "object": token,
            #     "user_id": user.id,
            # },
        )

        participants = User.objects.filter(
            program_enrollments__program__in=Program.objects.filter(
                Q(pre_activity=activity) |
                Q(post_activity=activity) |
                Q(sessions__pre_activity=activity) |
                Q(sessions__post_activity=activity) |
                Q(sessions__topic__pre_activity=activity) |
                Q(sessions__topic__post_activity=activity)
            )
        ).distinct()

        for participant in participants:
```

```
            send_subscription_and_notification(

subscription_module="program.graphql.Activity.subscriptions",
                subscription_class="ActivitySubscription",
                group=participant.username,
                subscription_payload={
                    "object": activity,
                    "mutation": "UPDATE",
                },
                # notification_event_type="survey_token_created",
                # notification_payload={
                #     "object": token,
                #     "user_id": user.id,
                # },
            )
        return UpdateActivity(activity=activity, success=True,
message="Activity updated successfully")
    except Activity.DoesNotExist:
        return UpdateActivity(activity=None, success=False,
message="Activity not found.")
    except Exception as e:
        return UpdateActivity(activity=None, success=False,
message=str(e))


class DeleteActivity(graphene.Mutation):
    class Arguments:
        id = graphene.ID(required=True)
```

```
    success = graphene.Boolean()
    message = graphene.String()

    @graphql_permission_required
    @staticmethod
    def mutate(root, info, id):
        try:
            activity = Activity.objects.get(pk=id)
            activity.delete()
            send_subscription_and_notification(

subscription_module="program.graphql.Activity.subscriptions",
                subscription_class="ActivitySubscription",
                group=info.context.user.username,
                subscription_payload={
                    "object": activity,
                    "mutation": "DELETE",
                },
                # notification_event_type="survey_token_created",
                # notification_payload={
                #     "object": token,
                #     "user_id": user.id,
                # },
            )
            return DeleteActivity(success=True, message="Activity deleted
successfully")
        except Activity.DoesNotExist:
            return DeleteActivity(success=False, message="Activity not
found")
```

```
class Mutation(graphene.ObjectType):
    create_activity = CreateActivity.Field()
    update_activity = UpdateActivity.Field()
    delete_activity = DeleteActivity.Field()

"""
Example:
mutation {
  createActivity(
    type: "quiz"
    description: "A quiz for the program"
    deadline: "2025-12-10T17:00:00Z"
    mappSurvey: 123
    is360: true
  ) {
    success
    message
    activity {
      id

      type
      description
      deadline
      mappSurvey
      is360

    }
```

```
  }
}

mutation {
  updateActivity(
    id: "5"
    type: "audio"
    description: "Updated description"
    deadline: "2025-12-15T10:30:00Z"
    mappSurvey: 456
    is360: false
  ) {
    success
    message
    activity {
      id

      type
      description
      deadline
      mappSurvey
      is360
    }
  }
}

mutation {
  deleteActivity(id: "5") {
    success
```

```
    message
  }
}
"""this is my program/graphql/activity/mutation.py
```

Yes — **this confirms the exact root cause**.
Your bug is in `DeleteActivity` **mutation**, specifically this part:

```python
activity = Activity.objects.get(pk=id)
activity.delete()
send_subscription_and_notification(
    ...
    subscription_payload={
        "object": activity,
        "mutation": "DELETE",
    },
)
```

✅ Exact reason for your error

After this line:

Python Run

you are still passing the **same Django model instance** into:

Python Run

That deleted `activity` object is now in a bad state for any code that tries to inspect **many-to-many fields** like:

- `activity.attachments`
- `activity.program_set`
- any reverse M2M / related field

So if your `send_subscription_and_notification` or GraphQL subscription serializer tries to access **attachments** (very likely, because you recently added them), Django throws:

> `"<Activity: test>" needs to have a value for field "id" before this many-to-many relationship can be used.`

🔥 Why first click fails, second click works

On first click:

1. Frontend detaches activity from program/session/topic
2. `DeleteActivity` runs
3. `activity.delete()` succeeds
4. `send_subscription_and_notification()` crashes because deleted object is passed
5. Mutation returns error to frontend

So:

- Activity is **actually deleted in DB**

- But mutation **throws after delete**
- Frontend thinks delete failed

Then second click:

- Object may already be gone / UI refreshed / state changed
- It appears to "work" or behaves differently

So yes — **delete is happening, but your post-delete notification crashes**.

✅ Exact fix (ready to paste)

Replace your `DeleteActivity` with this version

```
class DeleteActivity(graphene.Mutation):
    class Arguments:
        id = graphene.ID(required=True)

    success = graphene.Boolean()
    message = graphene.String()

    @graphql_permission_required
    @staticmethod
    def mutate(root, info, id):
        try:
            activity = Activity.objects.prefetch_related("attachments").get(pk=id)

            # IMPORTANT: collect everything needed BEFORE delete
            activity_id = activity.id
            activity_name = activity.name
            activity_type = activity.type
```

```
            activity_description = activity.description
            activity_deadline = activity.deadline
            activity_mapp_survey = activity.mapp_survey
            activity_is_360 = activity.is_360
            activity_mapp_survey_name = activity.mapp_survey_name
```

⚠️ But there is one important thing

This fix is **correct only if** your `send_subscription_and_notification()` and subscription code can handle a **dict** in `"object"`.

If your subscription code expects a real Django `Activity` instance, then passing a dict may break.

So the **safer immediate fix** is this:

✅ Best immediate safe fix (minimal change, guaranteed)

If you want the fastest fix with minimum risk, just **move the notification before delete**.

Replace only `DeleteActivity` with this:

```
class DeleteActivity(graphene.Mutation):
    class Arguments:
        id = graphene.ID(required=True)

    success = graphene.Boolean()
    message = graphene.String()
```

```
    @graphql_permission_required
    @staticmethod
    def mutate(root, info, id):
        try:
            activity = Activity.objects.prefetch_related("attachments").get(pk=id)

            # Send DELETE subscription BEFORE delete
            send_subscription_and_notification(
                subscription_module="program.graphql.Activity.subscriptions",
                subscription_class="ActivitySubscription",
                group=info.context.user.username,
                subscription_payload={
                    "object": activity,
                    "mutation": "DELETE",
```

✅ I strongly recommend THIS version first

Because:

- It requires **minimal code change**
- Keeps `"object": activity` as actual model instance
- Avoids breaking your subscription serialization
- Fixes the M2M-after-delete error immediately

🚨 Best production version (even better)

Actually, your current delete mutation only notifies:

Python Run

But in `UpdateActivity`, you notify:

- current user
- **all participants** whose program/session/topic contains that activity

So delete should also notify participants.

Better final version (recommended)

```
class DeleteActivity(graphene.Mutation):
    class Arguments:
        id = graphene.ID(required=True)

    success = graphene.Boolean()
    message = graphene.String()

    @graphql_permission_required
    @staticmethod
    def mutate(root, info, id):
        try:
            activity = Activity.objects.prefetch_related("attachments").get(pk=id)

            # Find participants BEFORE delete
            participants = User.objects.filter(
                program_enrollments__program__in=Program.objects.filter(
                    Q(pre_activity=activity) |
                    Q(post_activity=activity) |
                    Q(sessions__pre_activity=activity) |
                    Q(sessions__post_activity=activity) |
```

```
                    Q(sessions__topic__pre_activity=activity) |
```

✅ Why `prefetch_related("attachments")` helps

Even if subscription serialization touches:

```python
</> Python                                                    Run
```

because you send the subscription **before delete**, and because attachments are prefetched, it is safer and avoids lazy DB relation issues.

🔍 One more issue in your frontend (important)

Your frontend currently does:

1. Detach from program/session/topic
2. Then call `DELETE_ACTIVITY_MUTATION`

That means by the time delete mutation runs, this query inside `UpdateActivity`/future delete logic:

```python
Program.objects.filter(
    Q(pre_activity=activity) |
    Q(post_activity=activity) |
    ...
)
```

may return **nothing**, because you already removed the activity from structure.

So for delete, if you want to notify participants correctly, you have **two choices**:

Option A (best architecture)

Make backend delete mutation handle **both**:

- detach from structure
- delete activity

Option B (current architecture)

Keep frontend detach-first flow, but then participant lookup in delete mutation may not find anyone.

So if you care about participant delete subscriptions, the best design is:

- **Delete mutation should do full cleanup itself**
- Frontend should only call delete

✅ My practical recommendation for you RIGHT NOW

Step 1 (do this immediately)

Change `DeleteActivity` to send notification **before** `activity.delete()`.

This will fix your error.

Step 2 (optional but recommended)

In frontend, if backend delete is enough, simplify:

```
TypeScript
```

instead of:

- update parent
- then delete

But only do this if your backend model relationships are set up to cleanly remove M2M on delete.

🎯 Final answer in one line

Exact bug:

You are calling:

```
Python
Run
```

after `activity.delete()`, and the subscription code likely accesses `activity.attachments` (ManyToMany), which causes the error.

Exact fix:

Move `send_subscription_and_notification()` **before** `activity.delete()`.

✅ Paste-ready minimal fix

Use this exact replacement:

```
class DeleteActivity(graphene.Mutation):
    class Arguments:
        id = graphene.ID(required=True)

    success = graphene.Boolean()
    message = graphene.String()

    @graphql_permission_required
    @staticmethod
    def mutate(root, info, id):
        try:
            activity = Activity.objects.prefetch_related("attachments").get(pk=id)

            send_subscription_and_notification(
                subscription_module="program.graphql.Activity.subscriptions",
                subscription_class="ActivitySubscription",
                group=info.context.user.username,
                subscription_payload={
                    "object": activity,
                    "mutation": "DELETE",
                },
            )
```

If you want, send me your `program/graphql/Activity/subscriptions.py` (or wherever `ActivitySubscription` is defined) and I'll verify whether your subscription serializer is also touching `attachments` and give you the **best final version**.

```
class DeleteActivity(graphene.Mutation):
    class Arguments:
        id = graphene.ID(required=True)

    success = graphene.Boolean()
    message = graphene.String()

    @graphql_permission_required
    @staticmethod
    def mutate(root, info, id):
        try:
            activity =
Activity.objects.prefetch_related("attachments").get(pk=id)

            # IMPORTANT: collect everything needed BEFORE delete
            activity_id = activity.id
            activity_name = activity.name
            activity_type = activity.type
            activity_description = activity.description
            activity_deadline = activity.deadline
            activity_mapp_survey = activity.mapp_survey
            activity_is_360 = activity.is_360
            activity_mapp_survey_name = activity.mapp_survey_name
            activity_is_feedback = activity.is_feedback
            activity_mapp_tool = activity.mapp_tool
            attachment_ids = list(activity.attachments.values_list("id",
```

```
flat=True))

        # Delete from DB
        activity.delete()

        # Build a lightweight safe object/payload for subscription
        deleted_activity_payload = {
            "id": activity_id,
            "name": activity_name,
            "type": activity_type,
            "description": activity_description,
            "deadline": activity_deadline,
            "mapp_survey": activity_mapp_survey,
            "is_360": activity_is_360,
            "mapp_survey_name": activity_mapp_survey_name,
            "is_feedback": activity_is_feedback,
            "mapp_tool": activity_mapp_tool,
            "attachment_ids": attachment_ids,
        }

        send_subscription_and_notification(

subscription_module="program.graphql.Activity.subscriptions",
            subscription_class="ActivitySubscription",
            group=info.context.user.username,
            subscription_payload={
                "object": deleted_activity_payload,   # SAFE payload, not
deleted model instance
                "mutation": "DELETE",
```

```
            },
        )

        return DeleteActivity(success=True, message="Activity deleted
successfully")

    except Activity.DoesNotExist:
        return DeleteActivity(success=False, message="Activity not
found")
    except Exception as e:
        return DeleteActivity(success=False, message=str(e))
```

this cod eyou have given me

```
class DeleteActivity(graphene.Mutation):
    class Arguments:
        id = graphene.ID(required=True)

    success = graphene.Boolean()
    message = graphene.String()

    @graphql_permission_required
    @staticmethod
    def mutate(root, info, id):
        try:
            activity = Activity.objects.get(pk=id)
            activity_payload = {
                "id": activity.id,
                "name": activity.name,
            }
```

```
            activity.delete()
            send_subscription_and_notification(

subscription_module="program.graphql.Activity.subscriptions",
                subscription_class="ActivitySubscription",
                group=info.context.user.username,
                subscription_payload={
                    "object": activity_payload,
                    "mutation": "DELETE",
                },
                # notification_event_type="survey_token_created",
                # notification_payload={
                #     "object": token,
                #     "user_id": user.id,
                # },
            )
            return DeleteActivity(success=True, message="Activity deleted
successfully")
        except Activity.DoesNotExist:
            return DeleteActivity(success=False, message="Activity not
found")
```

this is another on my main branch i just want it to work like delete in single click for that what need add only that content in deleted_activity_payload = { this and give me fully updated final codeblock

Yes — for **single-click delete**, the **simplest and safest fix** is:

- keep your current structure
- **send a safe payload (dict) after delete**
- include enough fields so subscription/frontend won't break if it expects activity data

Since you asked specifically:

> what need add only that content in `deleted_activity_payload = {}` and give fully updated final codeblock

✅ Use this final `DeleteActivity` code

```
class DeleteActivity(graphene.Mutation):
    class Arguments:
        id = graphene.ID(required=True)

    success = graphene.Boolean()
    message = graphene.String()

    @graphql_permission_required
    @staticmethod
    def mutate(root, info, id):
        try:
            activity = Activity.objects.prefetch_related("attachments").get(pk=id)

            # Capture all needed fields BEFORE delete
            deleted_activity_payload = {
                "id": activity.id,
                "name": activity.name,
                "type": activity.type,
                "template_id": activity.template_id,
                "description": activity.description,
                "deadline": activity.deadline,
```

```
                    "mapp_survey": activity.mapp_survey
```

✅ What changed from your main branch version

Your main branch had only:

```python
activity_payload = {
    "id": activity.id,
    "name": activity.name,
```

I updated it to include:

- `type`
- `template_id`
- `description`
- `deadline`
- `mapp_survey`
- `is_360`
- `mapp_survey_name`
- `is_feedback`
- `mapp_tool`
- `attachment_ids`

This is the important part for your attachments integration and avoiding the M2M issue.

⚠️ Very important note

If your `ActivitySubscription` / `ProgramActivityType` **expects a real Django model instance**, then sending a dict may still cause issues (different error).

If after this you get a new error like:

- `'dict' object has no attribute ...`
- Graphene cannot resolve fields from dict

Then the **best fix** is to move notification **before delete** instead.

✅ Most reliable version (I recommend this if you want guaranteed working)

If you want the **highest chance of working immediately**, use this instead:

```
class DeleteActivity(graphene.Mutation):
    class Arguments:
        id = graphene.ID(required=True)

    success = graphene.Boolean()
    message = graphene.String()

    @graphql_permission_required
    @staticmethod
    def mutate(root, info, id):
        try:
            activity = Activity.objects.prefetch_related("attachments").get(pk=id)
```

```
            # Send subscription BEFORE delete so Graphene can safely read model fields + attach
            send_subscription_and_notification(
                subscription_module="program.graphql.Activity.subscriptions",
                subscription_class="ActivitySubscription",
                group=info.context.user.username,
                subscription_payload={
                    "object": activity,
                    "mutation": "DELETE",
```

✅ My honest recommendation for you

Since you said:

> i just want it to work like delete in single click

Use THIS ONE first (best for single-click success):

```
class DeleteActivity(graphene.Mutation):
    class Arguments:
        id = graphene.ID(required=True)

    success = graphene.Boolean()
    message = graphene.String()

    @graphql_permission_required
    @staticmethod
    def mutate(root, info, id):
        try:
            activity = Activity.objects.prefetch_related("attachments").get(pk=id)
```

```
            send_subscription_and_notification(
                subscription_module="program.graphql.Activity.subscriptions",
                subscription_class="ActivitySubscription",
                group=info.context.user.username,
                subscription_payload={
                    "object": activity,
                    "mutation": "DELETE",
                },
```

Why?

Because:

- no dict compatibility risk
- no M2M-after-delete issue
- least changes
- most likely to work immediately

Final answer in short

If you want only payload update:

Use the **first codeblock** (with full `deleted_activity_payload` dict).

If you want most reliable single-click delete:

Use the **second codeblock** (**send notification before delete**) ← **recommended**

If you want, next send me your `ActivitySubscription` **code** and I'll tell you **100% which of the two versions is correct** for your project.